BSc (Hons) in **Information Technology Specialized in AI**

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P – Performance Measure - Defines how the success of the agent is evaluated (the criteria used to judge its performance).
E – Environment - The external world in which the agent operates.
A – Actuators - The mechanisms the agent uses to take actions and affect the environment.
S – Sensors - The mechanisms the agent uses to perceive and gather information from the environment.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self.width - Represents the width of the grid environment.
self.height - Represents the height of the grid environment.
self.walls - Represents obstacles in the environment that the agent cannot pass through.
self.food_positions - Represents the locations of food items that the agent must collect.
self.opponents - Represents the positions of other agents (adversaries) moving within the environment.
self.agent_pos - Represents the agent's current physical location in the environment.
self.steps - Represents the progression of time/actions taken in the environment.
self.collision - Represents whether the agent has interacted with an opponent and the game environment has reached a collision state.
self.score - Represents the current state of the environment's reward/utility after interactions

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

The environment is Multi-Agent because the code initializes and manages multiple entities that can act within the same environment.

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of all the observations (percepts) that an agent has received from its environment since it started operating. It is a sequence of individual percepts collected over time, where each percept represents the information sensed by the agent at a specific moment. The agent uses this percept sequence to understand the state of the environment and make decisions about which actions to perform.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

The get_percept() method represents the Sensors (S) component of the PEAS framework.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

The environment is Fully Observable because the get_percept() method provides the agent with complete information about the important aspects of the environment state. The returned dictionary includes the agent's current position (agent_pos), all opponent positions (opponent_positions), food locations through the smells_food information, wall collisions (hit_wall), collision status, score, and remaining food count. Since the agent can access the relevant information needed to make decisions about its actions, it can observe the entire state of the environment. Therefore, this baseline environment is considered Fully Observable.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

When execute_action() deducts points for hitting a wall, the PEAS component being actively updated is the Performance Measure (P) component. The score is used to evaluate how well the agent performs in the environment. In this case, hitting a wall causes a penalty (self.score -= 5), which reduces the agent's performance score because the action was not successful.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is structurally important that the performance metric evaluates changes in the external environment state rather than the agent's internal processing effort because the purpose of an intelligent agent is to take actions that achieve goals in its environment. The PEAS framework measures success based on the outcomes of the agent's actions, such as collecting food, avoiding obstacles, or preventing collisions, rather than how the agent internally thinks or calculates. Evaluating external effects ensures that the agent is judged by its actual performance and effectiveness in interacting with the environment.

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical

> nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

If self.toxic_traps is added to the environment but intentionally hidden from the agent’s sensors, the environment changes from Fully Observable to Partially Observable. This is because the agent no longer has access to all the relevant information about the environment state. The toxic traps exist and can affect the agent’s score or survival, but the agent cannot detect their locations through get_percept(). As a result, the agent must make decisions without complete knowledge of the environment, making the environment Partially Observable.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

adding the 'smells_toxin' sensor expands the agent’s percept sequence by providing additional information about potential danger in the environment. This helps the agent act more rationally because it can use this new percept to make better decisions that maximize its expected utility. For example, if the agent detects a toxic trap at its current location or nearby through this sensor, it can avoid risky actions and choose safer paths instead. By considering more information about the environment, the agent can reduce penalties, avoid damage, and improve its chances of achieving its goal, making its actions more effective and utility-driven.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic "Vacuum World Exploit" example discussed in Lecture 01 illustrates that a poorly designed performance measure can cause an agent to behave in an unintended way. In the example, if the vacuum agent is rewarded only for reducing the amount of dirt, it may exploit the metric by repeatedly cleaning the same already-clean location or performing unnecessary actions instead of actually improving the environment. This shows the danger of faulty metrics, where an agent may maximize its score without truly achieving the intended goal. Therefore, the performance measure must be carefully designed so that the agent's behavior aligns with the actual objective.

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!

FACULTY OF COMPUTING